

目 录

1. 作品概要
2. 背景与需求
3. 总体设计
4. 数据与岗位知识库
5. 核心功能设计
6. 大模型与智能体
7. 前端与交互设计
8. 接口规范与安全合规
9. 部署与运维
10. 测试、评测与达标论证

作品概要

一、基本信息

项目	内容
命题编号	A13
命题方向	智能计算
题目类别	应用类
题目名称	基于 AI 的大学生职业规划智能体
作品名称	CareerAgent（工程仓库名：career-agent，工作区目录 vision-meet）
命题企业	明杉科技
学校	苏州城市学院
团队名称	【请填写团队名称】
指导教师	郑伊轮、纪其进
联系人/手机/邮箱	吴友桃//

二、一句话定位

本产品是一套面向高校学生的「岗位理解→能力画像→人岗匹配→职业路径→生涯报告→导出归档」全链路 Web 应用：以企业提供的计算机类岗位大数据为基础，结合大语言模型与智能体编排（PiCodingAgent）、向量检索（PostgreSQL+pgvector）与岗位路径图谱（Neo4j），帮助学生把碎片化自我认知与岗位要求对齐，输出可解释、可编辑、可导出的职业生涯发展材料。

三、行业背景与痛点

在数字经济与产业升级背景下，就业市场呈现「新兴领域缺人」与「部分专业就业难」并存的结构矛盾。大学生职业规划困难，往往源于：

- 1.自我认知模糊：兴趣、能力、性格与岗位匹配关系不清，易跟风考研、考公或「进大厂」，规划缺少个性化与可行性。
- 2.职业信息不对称：行业与岗位信息来源碎片化，对 AI、新能源等新兴岗位的真实技能与风险认知不足。

- 3.外部支持薄弱：高校生涯课偏理论与技巧，一线产业洞察不足；家庭引导两极化。
- 4.规划难落地：缺少实习与项目验证，面对行业快速变化缺少动态调整方法与评估指标。

本产品价值：在同一平台内完成岗位要求结构化拆解、个人能力结构化画像、多维度量化匹配与差距解释、结合图谱的发展路径展示以及分阶段行动计划与报告导出，降低「从信息到行动」的摩擦成本。

四、用户期望与产品回应

赛题用户期望	本产品摘要
快速了解就业市场对应届岗位的能力要求，并拆解具体能力要求	岗位导入与检索；岗位画像接口生成结构化要求（技能、证书、软素质等）；可结合知识库检索增强理解
清晰、准确分析学生就业能力与就业意愿	表单录入+简历上传双入口；输出结构化学生就业能力画像、完整度与竞争力评分及维度得分
给出可操作的生涯发展报告（方案+行动计划），扬长避短、精准匹配	四维人岗匹配（基础要求、职业技能、职业素养、发展潜力）+差距与建议；七章节职业报告（含路径与短中期计划）；支持结构化编辑与 PDF 一键导出

五、核心能力清单

5.1 就业岗位要求画像

- 构建不少于 10 个目标岗位的结构化画像（可随导入数据扩展至全量岗位）。
- 画像维度覆盖赛题要求方向，包括但不限于：专业技能、证书要求、创新能力、学习能力、抗压能力、沟通能力、实习/项目能力等（以接口返回字段与契约为准）。
- 支持对指定岗位生成/缓存复用画像，便于批量演示与复现。

5.2 岗位关联图谱与路径

- 垂直岗位图谱：展示岗位描述及晋升路径关联（promotion_routes）。
- 换岗路径图谱：相关岗位血缘/转换关系（transition_routes）。
- 赛题量化要求：至少 5 个岗位具备换岗规划，每个岗位不少于 2 条换岗路径（已在工程里程碑中落地，详见本文第三篇、第四篇）。

5.3 学生就业能力画像

- ◆ 数据来源：简历上传或表单自行录入。
- ◆ 输出：结构化能力画像、完整度评分、竞争力评分及与赛题方向一致的维度刻画。
- ◆ 大模型用于简历解析与信息抽取等环节（具体边界见本文第四篇、第五篇，避免与「规则评分」混淆）。

5.4 学生职业生涯发展报告

- 1) 职业探索与岗位匹配：结合岗位画像与学生画像，输出契合度与差距的量化与文字解释。
- 2) 职业目标与路径规划：报告内 `career_path` 章节与 Neo4j 图谱 API 联动，展示晋升与换岗路径。
- 3) 行动计划：短期/中期计划章节，含学习与实践安排建议；报告结构支持后续扩展「评估周期与指标」字段。
- 4) 编辑与导出：支持手动编辑各章节并保存；支持 PDF 导出与下载。

5.5 技术要求与指标

1. 大模型应用：以 PiCodingAgent 作为业务侧统一智能体入口，用于任务规划、工具编排及与 RAG (qmd+pgvector) 协同；岗位画像当前以规则基线为主并可与 Agent/检索增强演进衔接。
2. 可操作性、可解释性：匹配结果包含四维得分、加权总分、差距列表、改进建议与解释字段。
3. 准确性：依据仓库内 30 条手工抽样评测（20 条匹配+10 条画像），人岗匹配关键技能匹配准确率 80%、画像关键信息准确率 92%，综合判定达到赛题阈值（ $\geq 80\%$ / $>90\%$ ）。数据来源：
`data/evaluation/match.manual-samples.jsonl`、
`data/evaluation/profile.manual-samples.jsonl`，复算命令见本文第九篇。
4. 实用性：Vue3 前端流程页（首页、画像、匹配、报告、职业路径、Agent 等）与 RESTAPI 完整联调，适合课堂演示与答辩录屏。

六、技术架构亮点

- 单仓 Monorepo: `apps/frontend` (Vue3+TypeScript+Vite+Pinia+VueRouter)、`apps/backend` (Node.js+Express+TypeScript+Zod)、`packages/contracts` (OpenAPI+共享类型)，契约优先协作。

- 数据层：PostgreSQL 承载岗位、画像、学生、匹配、报告与导出记录；pgvector 承载知识库向量与混合检索；Neo4j 承载岗位路径图谱。
- 智能层：PiCodingAgent 统一编排；qmd 文档结构化切分+索引脚本（knowledge:index:*）支撑本地知识库建设（对应企业要求提交的「本地知识库资料」）。
- 工程质量：统一 Node22.12+；npmruntype-check 作为合入基线；接口版本化/api/v1。

七、创新点

- 1.可解释的人岗匹配：严格按赛题四维框架打分，并输出可追踪的说明与改进建议，而非单一「匹配度百分比」。
- 2.路径图谱与报告联动：图谱查询接口与报告 career_path 章节同一数据源逻辑，避免「报告与图谱各说各话」。
- 3.结构化报告+版本化：同一匹配结果可生成多版报告，便于迭代对比与答辩展示。
- 4.可演进架构：Repository 适配器模式，关系库/向量/图谱分层清晰，便于从演示环境扩展到院校私有化部署。

八、本文档内章节索引

原文件名/材料	并入位置
项目详细方案（主干）	第一篇～第六篇、第八篇及篇内各小节
项目技术架构	第二篇 2.6
RAG 优化方案	第五篇 5.6
接口文档	第七篇 7.6
API 端点速查	第七篇 7.7
环境变量摘要	第八篇 8.7
数据实体/四维评分细则	第四篇 4.10～4.11
手工抽样评测报告	第九篇 9.3
评测规范	第九篇 9.7

第一篇：背景与需求

1.1 宏观背景

我国正处于产业升级与数字经济深入发展的阶段，劳动力市场对高素质、复合型、具备持续学习能力的人才需求上升，同时部分传统专业供给与新兴岗位需求之间存在错配。大学生在职业规划上面临「信息过载但有效结构化信息不足」「自我认知与外部标准难以对齐」「规划停留在纸面难以迭代」等系统性困难。教育部与产业界持续推进就业指导数字化、精准化，企业命题方明杉科技在教育数智化领域积累了教务、学工、数据中台、一网通办与精准就业等产品经验，本题要求参赛队伍在智能计算方向交付可运行的 AI 职业规划智能体，具有明确的教学价值与产业落地想象空间。

1.2 问题陈述

赛题将学生侧痛点归纳为四类，本产品以软件功能逐条消解：

1.自我认知模糊、定位偏差：通过结构化学生画像、完整度与竞争力评分、与目标岗位四维对比，帮助学生把「自我感觉」转化为可对照、可改进的条目。

2.职业信息不对称：通过岗位大数据导入、岗位画像、职业路径图谱与可选知识库检索，把岗位要求与发展路径从「帖子与传说」还原为可查询、可解释的信息。

3.外部支持薄弱：本产品定位为 7×24 可复用的自助式智能体，在师资不足场景下仍可完成基础诊断与行动清单；输出报告可交由导师二次批注。

4.规划落地性差：报告内短期/中期计划章节与差距—行动映射，强调实习、项目与技能补齐路径；后续可扩展「评估周期与 KPI」字段以支持动态复盘。

1.3 建设目标

在赛事周期内交付可演示、可评测、可解释的端到端系统：

- (1) 业务闭环：岗位数据进入系统→岗位画像→学生画像（表单/简历）→人岗匹配→职业报告（多版本）→编辑→PDF 导出→职业路径可视化。
- (2) 技术闭环：Monorepo 工程+契约优先+PostgreSQL/pgvector/Neo4j+PiCodingAgent 编排+RAG 管线持续演进。
- (3) 质量闭环：手工抽样评测集+可复算脚本+Markdown 报告，对标赛题 80%/90%准确率门槛。

1.4 需求追溯矩阵

- (1) 就业岗位要求画像

赛题 a：不少于 10 个岗位画像；维度含专业技能、证书、创新能力、学习能力、抗压能力、沟通能力、实习能力等。

实现：jobs 模块；POST/api/v1/jobs/import 导入；
POST/api/v1/jobs/profile/generate 生成结构化画像；画像字段与契约类型 JobProfile 等对齐。

验收：导入≥10 条岗位记录后批量生成画像；抽检画像字段与职位描述一致性。

状态：已落地。

（2）岗位关联图谱

赛题 b-1 垂直图谱：岗位描述+晋升路径关联。

实现：career-path（或等价路由模块）；Neo4j 存储；GET/api/v1/career-paths/jobs/{job_id}返回 promotion_routes、nodes、edges。

验收：前端/career-paths 可展示晋升链。

状态：已落地。

赛题 b-2 换岗图谱：血缘关系；至少 5 个岗位，每岗≥2 条换岗路径。

实现：同一 API 返回 transition_routes；种子数据由 npmruncareer-path:sync（仓库根目录，实际执行 career-backend 脚本）同步。

验收：枚举 5 个规范岗位，逐岗核对≥2 条 transition_routes。

状态：已落地。

（3）学生就业能力画像

赛题 a：简历上传或自行录入；大模型拆解画像；完整度、竞争力评分。

赛题 b：维度与岗位侧同向（专业技能、证书、创新、学习、抗压、沟通、实习等）。

实现：profile 模块；POST/api/v1/profile、POST/api/v1/profile/resume；返回 completeness_score、competitiveness_score、dimension_scores 等。

验收：同一学生两种入口各跑一次，检查结构化字段与评分稳定合理。

状态：已落地。

（4）职业生涯发展报告

赛题 a 职业探索与匹配：人岗匹配度、契合与差距量化。

实现：matching+report；POST/api/v1/matches；报告章节 match_analysis 等。

状态：已落地。

赛题 b 目标与路径：行业趋势+岗位数据关联性+个人擅长+发展路径示例。

实现：报告 career_path+图谱 API；概述类内容可由模板与数据拼接，Agent 增强见第五篇。

状态：MVP 已落地。

赛题 c 行动计划：短期、中期；学习与实践；评估周期与指标。

实现：short_term_plan、mid_term_plan 章节；评估指标可在正文中结构化描述，独立字段可后续扩展。

状态：MVP 已落地。

赛题 d 编辑优化与导出：智能润色、完整性检查、手动编辑、一键导出。

实现：PATCH/api/v1/reports/{report_id}手动编辑；
POST/GET.../exports+GET.../downloadPDF。

规划：润色与完整性检查见概要介绍「边界」说明。

状态：编辑+PDF 已落地；润色/完整性检查/Docx 未作为默认交付。

（5）技术要求与指标

大模型：PiCodingAgent+业务工具+RAG 管线；禁止并行维护独立 ChatCompletions 业务主链路（工程约束）。

可解释匹配：四维打分+权重+说明。

准确率：见第九篇。

实用性：前端多页面联调完成。

第二篇：总体设计

2.1 设计原则

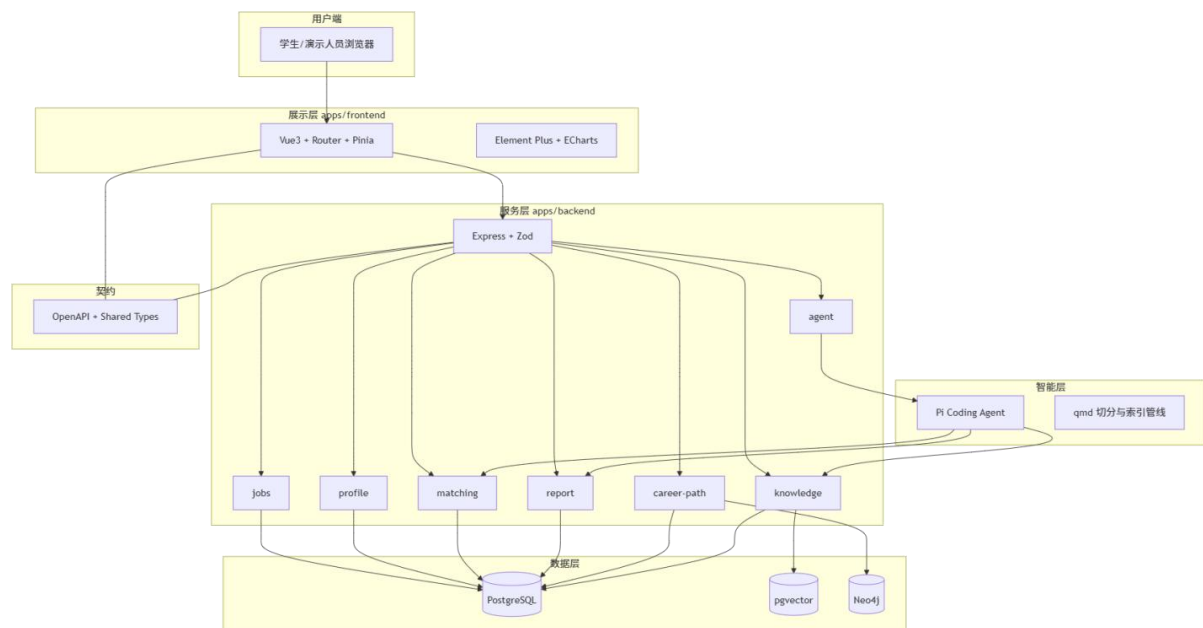
1.契约优先：接口与共享类型以 packages/contracts 为单一真源，变更顺序 OpenAPI→后端 schema/service→前端调用。

2.分层解耦：前端 app/features/shared；后端 route→service→repository→adapter。

3.可替换存储：业务代码依赖 repository 抽象，PostgreSQL 为当前主存储；向量与图数据库在适配器边界扩展。

4.稳定演示与可演进平衡：报告生成采用模板+结构化数据保证答辩稳定；LLM 与 Agent 在编排与检索侧逐步加深。

2.2 逻辑架构



2.3 模块边界说明

- **jobs**: 岗位导入、分页查询、岗位画像生成与缓存。
- **profile**: 学生画像创建（表单/简历）、列表查询。
- **matching**: 创建匹配、列表、详情、指纹缓存。
- **report**: 报告版本生成、查询、章节 PATCH、PDF 导出与下载。
- **career-path**: 按 `job_id` 查询规范角色、图节点边、晋升与换岗路径。
- **knowledge**: 知识库索引、混合检索、评测相关 API（见 OpenAPI）。
- **agent**: 任务创建、查询、对话别名等，对接 PiSDK。

2.4 关键业务时序

岗位导入：前端 `multipart` → `jobs.route` → `jobs.service` 解析清洗 → `repository.pg` 持久化 → 返回导入条数。

学生画像：JSON 或简历文件 → 校验 → 维度与完整度/竞争力计算 → 持久化 → 返回 `StudentProfileRecord`。

匹配与报告：选定 `student_profile_id` 与 `job_id` → 确保岗位画像版本 → 四维评分与解释 → 可选缓存 → 基于 `match_id` 生成报告新版本 → 用户 PATCH 编辑 → 触发 PDF 导出任务 → 下载。

2.5 Monorepo 目录

```
vision-meet/  
├── apps/  
│   ├── frontend/#Vue3 前端  
│   └── backend/#Express 后端  
├── packages/  
│   └── contracts/#OpenAPI+types  
├── infra/#docker-compose 等  
├── data/#原始数据、评测样本等  
├── docs/#文档  
└── scripts/
```

2.6 项目技术架构

适用仓库：/Users/peach/develop/career-agent

1.文档目标

本文件用于统一说明 CareerAgent 项目的技术架构基线，覆盖：

- Monorepo 工程组织方式。
- 前后端分层与模块边界。
- 契约优先（OpenAPI+共享类型）协作机制。
- 当前可运行架构与目标演进架构。

2.架构设计原则

- 1.契约优先：接口字段变更先改 packages/contracts，再改实现。
- 2.分层解耦：前端 app/features/shared，后端 route->service->repository->adapter。
- 3.模块内聚：业务能力按 domain 组织，避免横向散落。
- 4.可替换存储：结构化数据统一落 PostgreSQL，向量检索走 pgvector，图谱关系预留 Neo4j。
- 5.单仓协同：前后端与 contracts 统一在一个仓库中进行版本演进。
- 6.单一模型主链路：当前只允许 PiCodingAgent 作为业务运行时模型入口，禁止并行维护独立 ChatCompletions 业务链路。

3.技术栈总览

层次	技术选型	说明
前端	Vue3+TypeScript+Vite+Pinia+VueRouter	页面展示、交互、状态与路由管理
后端	Node.js+Express+TypeScript+Zod+Multer	RESTAPI、参数校验、文件上传
契约层	OpenAPI3.0+SharedTypeScriptTypes	前后端共享接口规范与数据类型
数据层（当前）	PostgreSQL+pgvector	结构化数据与知识检索统一落库
数据层（目标）	PostgreSQL+pgvector+Neo4j	在现有结构化/向量层基础上补齐图谱能力
基础设施	DockerCompose	本地拉起数据库与图数据库

4.Monorepo 工程架构

```
career-agent/  
├─apps/  
│ └─frontend/#Vue 前端 (app/features/shared)  
│ └─backend/#Express 后端 (modules/<domain>/shared)  
├─packages/  
│ └─contracts/  
│ └─openapi/#OpenAPI 契约  
│ └─types/#共享 TypeScript 类型  
├─infra/#基础设施编排 (docker-compose)  
├─data/#原始数据样本  
├─docs/#项目文档与规范  
├─scripts/#开发脚本  
└─services/#集成服务
```

5.总体分层架构

```
flowchart LR  
    U["用户/浏览器"] --> FE["前端应用 apps/frontend<br/>Vue3+Router+Pinia"]  
    FE --> API["后端 API apps/backend<br/>Express+Zod"]
```

```
API-->SVC["领域服务层<br/>Jobs/ProfileService"]
SVC-->REPO["Repository 抽象层"]
REPO-->ADAPTER1["PostgreSQLAdapter (当前)"]
REPO-->ADAPTER2["pgvectorAdapter (当前)"]
REPO-.演进.->ADAPTER3["Neo4jAdapter (目标)"]
FE<-->CONTRACT["packages/contracts<br/>OpenAPI+SharedTypes"]
API<-->CONTRACT
```

6. 前端架构 (apps/frontend)

6.1 分层职责

src/app: 应用装配层 (入口、全局挂载、路由装配)。

src/features/<feature>: 业务功能层 (页面、feature 路由)。

src/shared: 跨 feature 共享能力 (API 请求、工具函数、通用 UI)。

6.2 当前页面与路由

"/": dashboard 功能首页 (features/dashboard/pages/DashboardPage.vue)。

"/profile": profile 功能页 (features/profile/pages/ProfilePage.vue)。

6.3 前端与后端通信

API 基地址由 VITE_API_BASE_URL 配置, 默认 http://127.0.0.1:8000。

src/shared/api/jobs.ts 对接岗位查询。

src/shared/api/profile.ts 对接学生画像列表与创建。

请求/响应类型来自 @career/contracts/types, 避免重复定义。

7. 后端架构 (apps/backend)

7.1 分层职责

route: 协议层, 处理 HTTP 路由、状态码、请求/响应转换。

schemas: 参数校验 (Zod)。

service: 业务逻辑编排 (规则计算、流程控制)。

repository: 存储抽象接口。

repository.<adapter>: 具体数据源实现 (当前为 PostgreSQL/pgvector)。

7.2 已实现领域模块

1.jobs 域 (src/modules/jobs)

POST/api/v1/jobs/import: 导入岗位数据 (csv/tsv/json/xls/xlsx)。

GET/api/v1/jobs: 分页/关键词/行业查询岗位。

POST/api/v1/jobs/profile/generate: 生成岗位画像并支持缓存命中。

2.profile 域 (src/modules/profile)

GET/api/v1/profile: 查询学生画像列表。

POST/api/v1/profile: 创建学生画像并计算完整度、竞争力与维度得分。

7.3 应用入口与中间件

src/index.ts: 启动服务, 监听 PORT。

src/app.ts: 组装 CORS、JSONbodyparser、/healthz、模块路由与全局错误处理。

src/shared/config/env.ts: 统一环境变量读取与校验 (Zod)。

7.4 Agent 与模型边界

唯一业务模型入口: /api/v1/agent/*->PiCodingAgent。

兼容策略: 旧 /api/v1/agent/chat 仅作为协议别名, 内部直接映射到当前任务型 Agent。

报告生成策略: 职业报告当前固定使用模板生成器, 保证输出稳定、可复现、可编辑。

明确禁止: 不得在 shared/llm、独立 smoke 脚本或 report 模块中重新引入 OpenAIChatCompletions 直连运行链路, 避免仓库再次出现“双主链路”。

8. 契约层架构 (packages/contracts)

8.1 目录与职责

openapi/career-agent.openapi.yaml: API 协议定义。

types/index.ts: 前后端共享实体与请求/响应类型。

8.2 契约优先流程

1. 修改接口字段时先更新 OpenAPI 与共享类型。
2. 后端按新契约调整 schema/service/repository。
3. 前端按新契约更新 API 调用与页面渲染。
4. 统一执行 npmruntype-check 确认一致性。

9.数据架构

9.1 当前存储方案

PostgreSQL: 岗位、岗位画像、学生画像、匹配结果、报告、导出记录、Agent 任务快照。

pgvector: 知识库 chunk 向量、语义检索与召回评测。

特点: 支持并发访问、事务、查询过滤与审计留痕, 已满足当前联调与演示主链路。

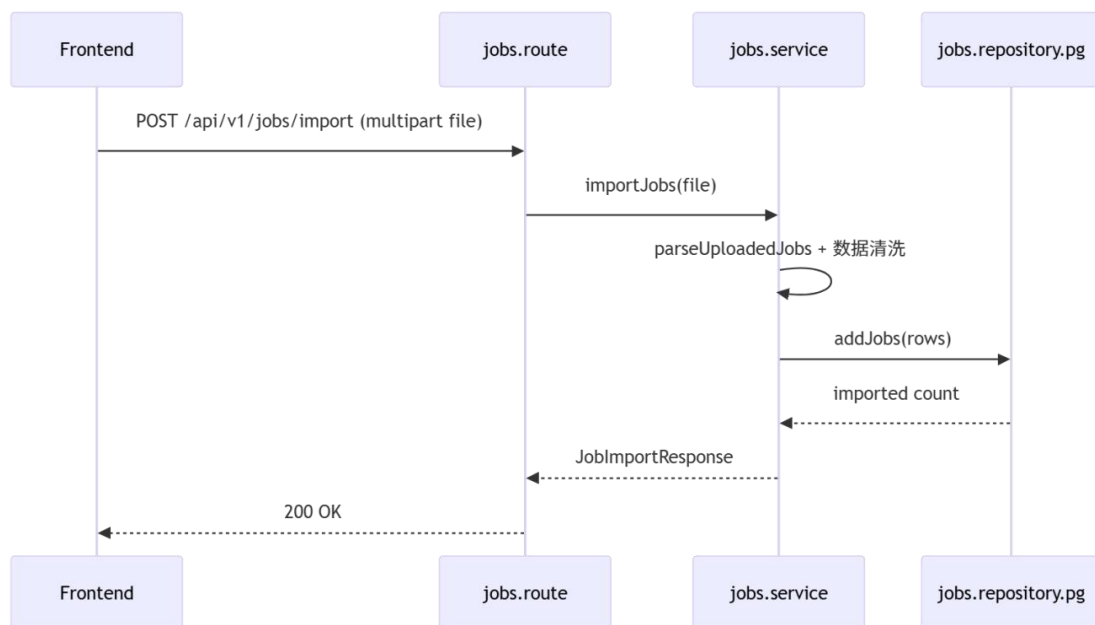
9.2 目标扩展方案

Neo4j: 岗位晋升路径、换岗路径、技能迁移关系图。

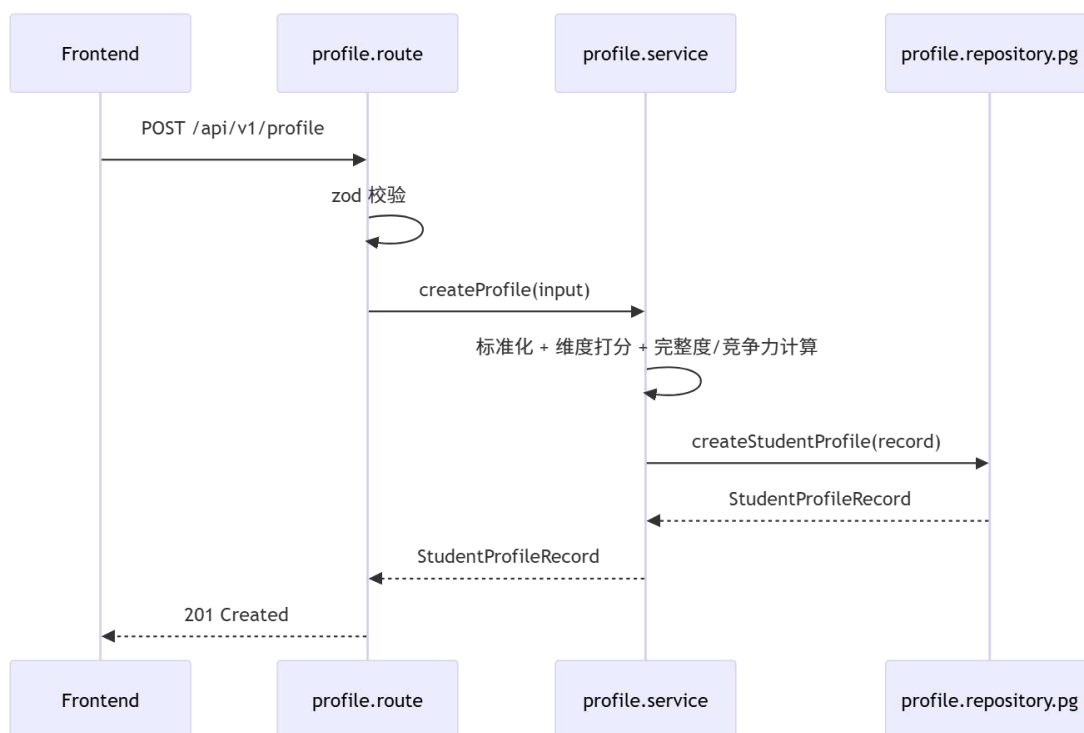
演进策略: 保持 service 层稳定, 在 repositoryadapter 边界继续扩展图谱能力。

10.关键运行链路

10.1 岗位导入链路



10.2 学生画像创建链路



11. 环境与部署架构

11.1 本地开发

根目录 npmrun dev 并行启动前后端。

前端默认: `http://127.0.0.1:5173`。

后端默认: `http://127.0.0.1:8000`。

infra/docker-compose.yml 可拉起:

postgres (pgvector 镜像)

neo4j

11.2 环境变量

后端: APP_ENV、PORT、PGHOST、PGPORT、PGDATABASE、PGUSER、PGPASSWORD。

前端: VITE_API_BASE_URL。

约束: 示例配置必须维护在 apps/backend/.env.example 与 apps/frontend/.env.example。

第三篇：数据与岗位知识库

3.1 企业岗位数据集

赛题说明企业提供约 10000 条、约 100 个岗位类别的计算机类信息化相关行业数据，字段包括：职位名称、工作地址、薪资范围、公司全称、所属行业、人员规模、企业性质、职位编码、职位描述、公司简介等。

本系统通过`POST/api/v1/jobs/import`支持 CSV/TSV/TXT/JSON/XLS/XLSX 等常见格式，便于对接企业网盘下发文件。

3.2 数据清洗策略

字段别名映射：不同表头（如「岗位名称」「职位名称」）归一到内部 schema。

编码与去噪：统一 UTF-8；去除非法控制字符；薪资、地址等可保留原文或结构化扩展字段（按实现版本）。

原始保留：在可行情况下保留原始 payload 或摘要，便于审计与后续 LLM 再分析。

3.3 本地知识库资料

建设目的：支撑「岗位以外的」行业政策、培养方案、技能学习路径、项目文档等 RAG 证据，使 Agent 或未来问答具备可引用片段，提升可解释性与专业性。

工程能力：

`npmrunknowledge:init`：初始化知识库相关库表/扩展。

`npmrunknowledge:index:jobs`：将岗位相关文本切块并向量化入库。

`npmrunknowledge:index:project-docs`：索引仓库内项目文档等。

`npmrunknowledge:eval`：检索质量基线评测。

技术要点：

分块策略：优先按标题层级切分，再按 token 上限二次切分。

元数据策略：记录来源文件、章节路径、岗位标签、时间戳。

召回策略：同问题并发多路检索（岗位库、行业库、案例库）；关键词+向量混合，可选重排序。

去重策略：相似 chunk 合并，避免上下文重复占用。

更新策略：增量更新索引，避免全量重建。

存储：pgvector 与 PostgreSQL 同行部署，降低运维复杂度。

3.4 规范岗位与 Neo4j 图谱数据

岗位原始数据条目标题多样，需映射到规范角色（**canonicalrole**）以构建可读的晋升/换岗网络。

归一化：canonical_role_key/canonical_role_title 与 job_id 的映射关系在后端维护。

同步脚本：npmruncareer-path:sync 将路径种子写入 Neo4j。

运行时：GET/api/v1/career-paths/jobs/{job_id} 聚合图数据与路径列表；可选 student_profile_id 为路径附加技能缺口提示。

第四篇：核心功能设计

4.1 岗位查询与画像生成

查询：GET/api/v1/jobs，支持 keyword、industry、offset、limit，用于学生浏览「市场侧」岗位列表。

画像生成：POST/api/v1/jobs/profile/generate，请求体含 job_id 与可选 force_regenerate。

缓存：若未强制重算且已有版本，返回 cached:true，节省演示时间与费用。

画像内容（逻辑维度）：硬技能列表、证书建议、软技能/通用素质、技能权重、摘要文本、置信度等。

当前策略：规则基线从职位描述抽取与归类，保证稳定；后续可接入 LLM 重述+RAG 证据引用提升丰富度与准确率。

4.2 职业路径图谱 API

路径：GET/api/v1/career-paths/jobs/{job_id}

查询参数：

student_profile_id：用于个性化缺口技能摘要。

depth（可选，1-3）：子图深度，默认 2。

响应核心：

nodes/edges：供前端 ECharts 力导向或关系图渲染。

`promotion_routes`: 垂直晋升序列。

`transition_routes`: 换岗路径，每岗多条。

错误码示例: `JOB_NOT_FOUND`、`CAREER_PATH_ROLE_NOT_MAPPED`、`CAREER_PATH_GRAPH_UNAVAILABLE`，便于演示时排查配置。

4.3 学生画像：表单录入

路径: `POST/api/v1/profile`

典型字段: 姓名、`target_role`、技能列表、`personal_summary` 等。

输出: `dimension_scores`、`completeness_score`、`competitiveness_score`、`summary`、`source_type` 等。

设计意图: 快速人工录入适合课堂无简历文件的演示；字段与后续匹配维度对齐。

4.4 学生画像：简历上传

路径: `POST/api/v1/profile/resume (multipart/form-data)`

字段: `file` 必填; `target_role` 必填; `name` 可选; `parse_mode: strict|tolerant`。

说明: 简历解析链路可能调用大模型或解析服务做实体抽取; `tolerant` 模式在格式不完美时尽量返回可用结构，避免演示失败。

4.5 人岗匹配：四维框架

路径: `POST/api/v1/matches`

维度:

- 1.基础要求: 学历、专业背景、基础门槛类匹配。
- 2.职业技能: 硬技能与岗位技能权重对比。
- 3.职业素养: 沟通、协作、责任与稳定性等。
- 4.发展潜力: 学习成长性、可迁移技能、上限空间等。

输出:

`dimension_scores`: 各维得分。

`total_score`: 按可配置权重加权后的综合分。

`gaps`: 缺口列表。

suggestions: 可执行建议。

explanations: 文字解释，服务「可解释性」评分点。

from_cache/input_fingerprint: 同一输入复现，便于评测与答辩。

列表与详情: GET/api/v1/matches、GET/api/v1/matches/{match_id}。

4.6 职业生涯发展报告

生成: POST/api/v1/reports, 请求体{"match_id":<id>}, 每次生成新版本 (version 递增)。

固定七章 (sections, key 唯一):

- 1.overview: 总览。
- 2.match_analysis: 匹配分析。
- 3.strengths: 优势。
- 4.gaps_and_actions: 差距与行动。
- 5.career_path: 路径 (与图谱叙事一致)。
- 6.short_term_plan: 短期计划。
- 7.mid_term_plan: 中期计划。

编辑: PATCH/api/v1/reports/{report_id} 必须提交完整 7 章结构, 避免部分更新导致状态不一致。

导出 PDF: POST/api/v1/reports/{report_id}/exports, format:pdf; 列表
GET.../exports; 下载 GET/api/v1/report-exports/{export_id}/download。

4.7 编辑优化、润色与完整性

子需求	实现状态	说明
手动编辑	已支持	PATCH 保存结构化章节
一键导出	已支持 (PDF)	Docx 为后续增强
智能润色	规划中	建议: Agent 生成修订稿+用户 diff 确认+写回章节
完整性检查	规划中	建议: Zod/OpenAPI 级章节非空校验、关键字段检查表、可读性提示

4.8 与 Agent 的协同边界

Agent 可通过工具读取匹配结果、检索命中、报告模板，执行「规划—检索—总结」任务；默认报告生成仍以模板为主保证稳定性（与第二篇 2.6 中「Agent 与模型边界」所述报告生成策略一致）。演进路线是按章节逐步 LLM 化，每次变更配套评测回归。

4.9 核心数据实体说明

Job（岗位原始记录）

对应导入行：标题、描述、行业、薪资、公司信息等。

与 job_id 主键关联画像、匹配、图谱映射。

JobProfile（岗位画像快照）

结构化技能、证书、软素质、权重、摘要、置信度、profile_version。

匹配时记录 job_profile_version 以便复现「当时依据的岗位要求版本」。

StudentProfile（学生画像）

source_type 区分表单/简历；source_digest 等用于去重或溯源（若启用）。

dimension_scores 与 completeness_score、competitiveness_score 服务赛题「完整度、竞争力」。

MatchResult（匹配结果）

四维分数、加权总分、gaps、suggestions、explanations。

scoring_version：评分规则版本化，支撑评测与回归。

input_fingerprint：输入指纹，用于缓存与科学对比实验。

CareerReport（职业报告）

与 match_id 绑定；version 递增；sections 为七章数组元素，每元素含 key、title、content（或等价结构）。

ReportExport（导出记录）

format（如 pdf）、file_name、file_size_bytes、download_path 或等价下载标识。

KnowledgeChunk（知识块）

chunk_text、embedding、metadata（来源、章节、标签）。

服务 RAG 证据链。

AgentTask（智能体任务快照）

计划步骤、工具调用轨迹、警告信息、与业务结果引用；用于审计与答辩演示「智能体如何工作」。

4.10 人岗匹配四维评分细则

1. 基础要求

关注点：学历层次、专业对口度、硬性门槛（如证书、年限）是否在简历/表单中有对应证据。

常见失分原因：目标岗位为后端开发但简历无语言与框架证据；学历字段缺失导致保守打分。

2. 职业技能

关注点：岗位画像中的硬技能与学生技能列表、项目经历的交集与深度。

可解释输出：「岗位强调分布式与 K8s，学生仅有单机部署经验」→明确进入 gaps 与 suggestions。

3. 职业素养

关注点：沟通协作、文档习惯、责任心、抗压（可从项目描述、角色、社团经历推断；证据不足时降置信度）。

赛题对齐：避免纯关键词匹配，结合解释字段说明推断依据。

4. 发展潜力

关注点：学习能力信号（自学课程、竞赛、开源）、可迁移技能、成长轨迹。

用途：区分「当下完全匹配」与「短期可培养」候选人，符合企业校招真实决策。

综合分

对各维得分按岗位可配置权重加权求和，权重可随岗位类别调整（如研发岗抬高职业技能权重）。输出中同时保留维度分与总分，满足「可解释性」评分点。

第五篇：大模型与智能体

5.1 选型与角色分工

- PiCodingAgent：业务运行时统一智能体入口（/api/v1/agent/*），负责任务拆解、工具调用与结果汇总。
- qmd+pgvector：文档切分、向量索引与混合检索，为 Agent 提供证据片段。
- 岗位画像/报告默认链路：规则与模板为主；LLM 用于简历解析、Agent 总结等明确边界的能力，避免「全文黑盒生成」导致答辩不可控。

5.2 禁止双主链路的工程原因

若同时存在直连 ChatCompletions 与 PiAgent 两套业务入口，会导致：提示词分叉、结果不可比、密钥与审计分散、类型契约难以统一。本仓库架构文档明确仅保留 Pi 作为业务模型主入口；历史别名 /api/v1/agent/chat 映射到当前任务型 Agent，而非并行体系。

5.3 Agent 任务与工具

结合 OpenAPI 中 Agent 相关 schema，任务步骤可包含（示例枚举，以契约为准）：任务规划、上下文拉取、知识检索、匹配评价、报告生成、最终回答等。工具层应对接：knowledge/search、匹配创建/查询、报告读写等受控能力，并记录步骤轨迹供审计与失败重试。

5.4 失败降级与可观测性

检索为空：降级为规则摘要或提示用户补充关键词。

模型不可用：报告仍可通过模板生成；Agent 接口返回明确错误码。

自检：npmrunagent:smoke（在 career-backend）用于环境连通性最小验证。

5.5 RAG 质量目标

检索与回答质量的量化指标见第五篇 5.6 第六节：Recall@K、MRR、事实一致性、证据覆盖率、可解释性评分，以及业务指标关键技能匹配准确率 $\geq 80\%$ 、画像关键信息准确率 $> 90\%$ 、任务成功率与重试成功率等，并与迭代联动。

5.6 RAG 优化方案（Pi+qmd）

1.目标

- 1) 提升岗位知识与职业规划知识的检索召回质量。
- 2) 让大模型回答具备证据引用与可解释性。
- 3) 通过 PiCodingAgent 管理多阶段任务，降低链路不稳定性。

2.技术选型

- (1) 智能体编排：PiCodingAgent。
- (2) 文档处理：qmd（主），Marker/PyMuPDF（补充解析）。

(3) 向量检索: PostgreSQL+pgvector。

(4) 结构化存储: PostgreSQL。

3.端到端链路

- a. 数据输入: 岗位数据、行业资料、政策资料、课程资料、简历文本。
- b. 文档预处理: 去噪、规范编码、章节识别、表格文本化。
- c. qmd 结构化切分: 按语义段落和标题层级生成 chunk。
- d. 向量化入库: 保存 chunk_text+metadata+embedding。
- e. 检索阶段: 关键词召回+向量召回+重排序。
- f. Agent 编排: PiCodingAgent 串联检索、分析、报告生成。
- g. 输出阶段: 回答内容+证据片段+置信度。

4.qmd 优化策略

1. 分块策略: 优先按标题层级切分, 再按 token 上限二次切分。
2. 元数据策略: 记录来源文件、章节路径、岗位标签、时间戳。
3. 召回策略: 同问题并发多路检索(岗位库、行业库、案例库)。
4. 去重策略: 相似 chunk 合并, 避免上下文重复占用。
5. 更新策略: 增量更新索引, 避免全量重建。

5.PiCodingAgent 编排职责

- A. 查询意图识别: 区分岗位问答、能力诊断、路径规划、报告生成。
- B. 工具编排: 触发检索、评分、报告模板、导出等工具。
- C. 证据约束: 回答必须绑定检索证据片段。
- D. 失败重试: 检索为空或证据不足时自动降级与重试。
- E. 审计日志: 记录每次工具调用、耗时、失败原因。

6.评测指标

检索召回: Recall@K、MRR。

回答质量：事实一致性、证据覆盖率、可解释性评分。

业务指标：关键技能匹配准确率>=80%，画像关键信息准确率>90%。

稳定性：任务成功率、平均响应时长、重试成功率。

7.实施顺序

- 1. 完成 qmd 文档切分和入库脚本。
- 2. 接入 pgvector 检索和评测脚本。
- 3. 接入 PiCodingAgent 编排层。
- 4. 打通“检索->匹配->报告”全链路。
- 5. 完成 A/B 评测并固化最优参数。

第六篇：前端与交互设计

6.1 技术栈

- 框架：Vue3+TypeScript+Vite。
- 状态与路由：Pinia、VueRouter。
- UI：ElementPlus。
- 图形：ECharts（职业路径可视化）；架构目标中曾含 G6，以实际依赖为准。

6.2 工程分层

- src/app：入口、全局样式、路由装配。
- src/features/<feature>：业务内聚页面与 feature 级路由。
- src/shared：api、工具、通用组件。

6.3 页面与路由

路径	功能模块	用户任务
/	dashboard	总览、入口导航、数据导入相关入口（以实际页面为准）

路径	功能模块	用户任务
/profile	profile	表单/简历创建学生画像
/matching	matching	选择画像与岗位，发起匹配，跳转报告或路径
/report	report	查看/编辑报告版本，导出 PDF
/career-paths	career-path	查看岗位图谱与晋升、换岗路径
/agent	agent	智能体任务演示

6.4 可用性与无障碍

表单校验错误与 `APIVALIDATION_ERROR` 映射为用户可读中文。

关键操作（匹配、导出）提供加载态与失败重试入口。

色弱对比度遵循 `ElementPlus` 默认主题；若校赛有要求可再加强。

第七篇：接口规范与安全合规

7.1 REST 约定

Base: 默认 `http://127.0.0.1:8000`（本地）。

版本前缀: `/api/v1`。

Content-Type: `JSON`；上传接口使用 `multipart/form-data`。

7.2 统一错误结构

```
{
  "code": "VALIDATION_ERROR",
  "message": "请求参数不合法",
  "detail": {},
  "trace_id": "可选 UUID"
}
```

7.3 与赛题强相关的端点索引

- 岗位: `/api/v1/jobs/import`、`/api/v1/jobs`、`/api/v1/jobs/profile/generate`
- 画像: `/api/v1/profile`、`/api/v1/profile/resume`
- 匹配: `/api/v1/matches`、`/api/v1/matches/{match_id}`
- 报告: `/api/v1/reports` 系列、`/api/v1/report-exports/{export_id}/download`
- 路径: `/api/v1/career-paths/jobs/{job_id}`
- 知识库: `/api/v1/knowledge/*` (索引、检索、评测)
- Agent: `/api/v1/agent/*`
- 健康检查: `GET/healthz`

7.4 配置与密钥

后端环境变量集中校验 (apps/backend 内 `shared/config/env.ts` 等)。

必须维护 `apps/backend/.env.example`、`apps/frontend/.env.example`, 禁止将真实密钥提交仓库。

数据库、Neo4j、模型服务密钥仅通过环境变量注入。

7.5 隐私与合规

简历与画像属个人敏感信息, 演示数据集应脱敏 (虚构姓名、电话、邮箱)。

生产/试点部署建议: 访问控制、审计日志、数据留存策略与学校信息化规范对齐。

本方案默认院校本地化部署友好, 不上传学生数据至不可信第三方 (取决于实际 `.env` 配置, 需在部署文档中声明)。

7.6 CareerAgent 接口文档

1. 文档说明

本文件用于提供 CareerAgent 当前后端 RESTAPI 的人读版说明, 便于前后端联调、接口验收和演示准备。

1. OpenAPI: `packages/contracts/openapi/career-agent.openapi.yaml`

2. 共享类型: `packages/contracts/types/index.ts`

接口字段变更时，必须先更新 `contracts`，再同步本文件。

2.基本信息

BaseURL: `http://127.0.0.1:8000`

接口风格: `/api/v1` 版本化 RESTAPI

返回格式: `application/json`

3.通用错误响应

除上传文件的 `multipart` 请求外，所有接口均返回统一 JSON 错误结构：

```
{
  "code": "VALIDATION_ERROR",
  "message": "请求参数不合法",
  "detail": {},
  "trace_id": "2b54cf0c-xxxx-xxxx-xxxx-xxxxxxxxxxxxx"
}
```

字段说明：

- 1. `code`：错误码，供前端稳定判断错误语义。
- 2. `message`：面向用户或调用方的错误说明。
- 3. `detail`：可选，通常用于字段级校验错误。
- 4. `trace_id`：可选，请求追踪标识，便于排障。

4.接口总览

模块	方法	路径	说明
Health	GET	<code>/healthz</code>	服务健康检查
Jobs	POST	<code>/api/v1/jobs/import</code>	导入岗位数据文件
Jobs	GET	<code>/api/v1/jobs</code>	查询岗位列表

模块	方法	路径	说明
Jobs	POST	/api/v1/jobs/profile/generate	生成岗位画像
Profile	GET	/api/v1/profile	查询学生画像列表
Profile	POST	/api/v1/profile	手动创建学生画像
Profile	POST	/api/v1/profile/resume	通过简历上传创建学生画像
Matching	POST	/api/v1/matches	创建人岗匹配结果
Matching	GET	/api/v1/matches	查询匹配结果列表
Matching	GET	/api/v1/matches/{match_id}	查询匹配结果详情
CareerPath	GET	/api/v1/career-paths/jobs/{job_id}	查询岗位图谱与路径规划
Report	POST	/api/v1/reports	基于匹配结果生成职业报告版本
Report	GET	/api/v1/reports?match_id=	查询某个匹配结果下的报告版本列表
Report	GET	/api/v1/reports/{report_id}	查询职业报告详情
Report	PATCH	/api/v1/reports/{report_id}	保存职业报告结构化章节编辑结果
Report	POST	/api/v1/reports/{report_id}/exports	生成并登记 PDF 导出产物
Report	GET	/api/v1/reports/{report_id}/exports	查询当前报告版本的导出记录
ReportExport	GET	/api/v1/report-exports/{export_id}/download	下载已生成的 PDF 文件

5.Health

GET/healthz

说明：返回服务状态与当前数据存储路径。

成功响应示例：

```
{
  "status":"ok",
  "env":"dev",
  "store":"/path/to/data.json"
}
```

6.Jobs

POST/api/v1/jobs/import

说明：上传 CSV/TSV/TXT/JSON/XLS/XLSX 岗位文件并导入。

请求类型：multipart/form-data

请求字段：

- 1.file：必填，岗位数据文件。

成功响应示例：

```
{
  "imported":10,
  "skipped":0,
  "message":"导入完成"
}
```

GET/api/v1/jobs

说明：按关键字、行业和分页条件查询岗位列表。

Query 参数：

- 1.keyword：可选，岗位关键字。

2. industry: 可选，行业筛选。
3. offset: 可选，默认 0。
4. limit: 可选，默认 20，最大 100。

成功响应字段：

1. total: 总记录数。
2. items: 岗位数组，元素类型为 JobRecord。

POST/api/v1/jobs/profile/generate

说明：为指定岗位生成结构化岗位画像；若已有画像则直接返回最新快照。

请求体：

```
{
  "job_id":1001,
  "force_regenerate":false
}
```

成功响应核心字段：

1. job_id
2. profile_version
3. hard_skills
4. certificates
5. soft_skills
6. skill_weights
7. summary
8. confidence
9. cached

7.Profile

GET/api/v1/profile

说明：查询学生画像列表。

成功响应字段：

1. total
2. items：学生画像数组，元素类型为 StudentProfileRecord。

POST/api/v1/profile

说明：通过手动录入创建学生画像。

请求体核心字段：

```
{
  "name": "张三",
  "target_role": "前端开发工程师",
  "skills": ["TypeScript", "Vue"],
  "personal_summary": "具备组件化开发经验"
}
```

成功响应核心字段：

1. id
2. source_type
3. source_digest
4. dimension_scores
5. completeness_score
6. competitiveness_score
7. summary

POST/api/v1/profile/resume

说明：通过简历文件上传创建学生画像。

请求类型：multipart/form-data

请求字段：

1. file：必填，简历文件。
2. target_role：必填，目标岗位。
3. name：可选，学生姓名。
4. parse_mode：可选，strict|tolerant，默认 tolerant。

成功响应结构与手动创建画像一致。

8.Matching

POST/api/v1/matches

说明：基于学生画像和岗位生成一条人岗匹配结果。

请求体：

```
{
  "student_profile_id":1,
  "job_id":1001,
  "force_recalculate":false
}
```

成功响应核心字段：

1. id
2. student_profile_id
3. job_id
4. job_profile_version
5. scoring_version
6. input_fingerprint
7. from_cache
8. dimension_scores
9. total_score
10. gaps
11. suggestions
12. explanations

GET/api/v1/matches

说明：按学生画像、岗位、分页条件查询匹配结果摘要列表。

Query 参数：

1. student_profile_id: 可选。
2. job_id: 可选。
3. offset: 可选，默认 0。

4. limit: 可选, 默认 20, 最大 100。

成功响应字段:

1. total

2. items: 匹配结果摘要数组, 元素类型为 MatchResultSummary。

GET/api/v1/matches/{match_id}

说明: 查询单条匹配结果详情。

Path 参数:

1. match_id: 必填, 匹配结果 ID。

成功响应结构与 POST/api/v1/matches 返回结构一致。

常见错误码:

1. MATCH_NOT_FOUND

2. JOB_NOT_FOUND

3. STUDENT_PROFILE_NOT_FOUND

9.Report

POST/api/v1/reports

说明: 基于既有 match_id 生成一份新的职业报告版本。

请求体:

```
{
  "match_id":1
}
```

成功响应核心字段:

1. id

2. match_id

3. version

4. student_profile_id

5. job_id

6. total_score

7. sections

8. created_at

9. updated_at

章节固定为以下 7 个：

1) Overview

2) match_analysis

3) Strengths

4) gaps_and_actions

5) career_path

6) short_term_plan

7) mid_term_plan

GET/api/v1/reports

说明：按 match_id 查询该匹配结果下的所有报告版本。

Query 参数：

1. match_id: 必填，匹配结果 ID。

成功响应字段：

1. total

2. items: 报告摘要数组，按 versiondesc 排列。

GET/api/v1/reports/{report_id}

说明：查询单份职业报告详情。

Path 参数：

1. report_id: 必填，报告 ID。

成功响应字段：

1. id

2. match_id

3. version

4. student_profile_id

5. job_id

6. total_score

7. sections

8. created_at

9. updated_at

PATCH/api/v1/reports/{report_id}

说明：保存当前报告版本的结构化章节编辑结果，不创建新版本。

请求体示例：

```
{
  "sections": [
    {
      "key": "overview",
      "title": "报告摘要",
      "content": "当前整体匹配度较高，建议保持优势并补齐短板。"
    }
  ]
}
```

常见错误码：

1. REPORT_NOT_FOUND

2. REPORT_SECTION_INVALID

POST/api/v1/reports/{report_id}/exports

说明：为指定报告版本生成一份新的 PDF 导出产物，并立即返回可下载记录。

Path 参数：

1. report_id: 必填，报告 ID。

请求体：

```
{
  "format": "pdf"
}
```

成功响应核心字段：

- 1.id
- 2.report_id
- 3.format
- 4.file_name
5. file_size_bytes
- 6.created_at
- 7.download_path

常见错误码：

- 1.REPORT_NOT_FOUND
- 2.REPORT_EXPORT_GENERATION_FAILED

GET/api/v1/reports/{report_id}/exports

说明：查询当前报告版本的历史导出记录列表。

Path 参数：

- 1.report_id: 必填，报告 ID。

成功响应字段：

- 1.total
- 2.items: 导出记录数组，按导出时间倒序排列。

GET/api/v1/report-exports/{export_id}/download

说明：下载已生成的 PDF 文件。

Path 参数：

- 1.export_id: 必填，导出记录 ID。

成功响应：

- 1.Content-Type:application/pdf
- 2.Content-Disposition:attachment

10.CareerPath

GET/api/v1/career-paths/jobs/{job_id}

说明：基于岗位 ID 返回规范岗位图谱、首批晋升/换岗路径，以及可选的学生个性化技能缺口摘

要。

Path 参数:

1. job_id: 必填, 岗位 ID。

Query 参数:

1. student_profile_id: 可选, 学生画像 ID; 传入后会对每条路径补充缺口技能。
2. depth: 可选, 图谱深度, 范围 1-3, 默认 2。

成功响应核心字段:

1. job_id
2. job_title
3. student_profile_id
4. canonical_role_key
5. canonical_role_title
6. target_node_id
7. nodes
8. edges
9. promotion_routes
10. transition_routes

常见错误码:

1. JOB_NOT_FOUND
2. CAREER_PATH_ROLE_NOT_MAPPED
3. CAREER_PATH_GRAPH_UNAVAILABLE

11. 当前范围说明

当前已实现接口能力覆盖:

- ① 岗位导入、查询与岗位画像生成。
- ② 学生画像创建与查询。
- ③ 人岗匹配创建、列表、详情与复现缓存。
- ④ 职业报告生成、版本列表、详情、结构化编辑保存与 PDF 导出。
- ⑤ 职业路径图谱查询（首轮 Neo4j 闭环）。

- ⑥ 报告润色与完整性检查
- ⑦ LLM 实际生成报告正文

7.7 相关 API 端点索引

系统与健康

GET/healthz: 服务存活探测; 联调与运维巡检入口。

岗位 jobs

POST/api/v1/jobs/import: multipart 上传岗位数据文件, 返回导入/跳过计数

GET/api/v1/jobs: 分页与筛选列表, 支撑「浏览就业市场岗位」场景。

POST/api/v1/jobs/profile/generate: 单岗位画像生成或缓存命中返回。

学生画像 profile

GET/api/v1/profile: 画像列表, 便于管理多次录入/多角色目标。

POST/api/v1/profile: 结构化表单创建。

POST/api/v1/profile/resume: 简历文件创建。

人岗匹配 matching

POST/api/v1/matches: 创建一条匹配结果 (四维+总分+差距+建议+解释)。

GET/api/v1/matches: 条件过滤列表。

GET/api/v1/matches/{match_id}: 详情, 用于报告生成前核对。

职业报告 report

- POST/api/v1/reports: 按 match_id 新建报告版本。
- GET/api/v1/reports?match_id=: 同一匹配下多版本列表。
- GET/api/v1/reports/{report_id}: 报告详情 (七章 sections)。
- PATCH/api/v1/reports/{report_id}: 整包保存七章编辑结果。
- POST/api/v1/reports/{report_id}/exports: 触发 PDF 导出。
- GET/api/v1/reports/{report_id}/exports: 导出历史。

- GET/api/v1/report-exports/{export_id}/download: 下载 PDF 二进制流。

职业路径 career-path

GET/api/v1/career-paths/jobs/{job_id}: 图谱节点边+晋升路径+换岗路径; 可选学生画像 ID。

知识库 knowledge

POST/api/v1/knowledge/index: 索引任务提交。

POST/api/v1/knowledge/search: 混合检索。

POST/api/v1/knowledge/evaluations: 检索评测任务。

智能体 agent

POST/api/v1/agent/tasks: 创建编排任务。

GET/api/v1/agent/tasks/{task_id}: 任务状态与结果。

POST/api/v1/agent/chat: 兼容别名, 内部指向统一任务型 Agent。

第八篇：部署与运维

8.1 运行环境

Node.js: $\geq 22.12.0 < 23$ (见根目录 package.json engines 与 .nvmrc/.node-version)。

包管理: 仓库根目录 npm install。

8.2 本地开发

```
npm run dev
```

并行启动前端 (默认 Vite 端口如 5173) 与后端 (默认 PORT 如 8000)。亦可:

```
npm run dev:backend  
npm run dev:frontend
```

8.3 数据库与中间件

infra/docker-compose.yml: 拉起 PostgreSQL (pgvector 镜像)、Neo4j 等。

后端连接参数: PGHOST、PGPORT、PGDATABASE、PGUSER、PGPASSWORD 等。

8.4 前端 API 基地址

环境变量 VITE_API_BASE_URL, 默认指向 http://127.0.0.1:8000。

8.5 构建与类型检查

```
npm run type-check  
npm run build
```

8.6 运维演进

结构化日志、请求 ID 贯通、基础指标 (QPS、错误率、P95 延迟)。

CI 中强制执行 type-check 与 format:check。

数据库迁移版本化管理。

8.7 环境变量与本地配置

后端:

- APP_ENV: 运行环境标识 (如 dev)。
- PORT: HTTP 监听端口。
- PGHOST/PGPORT/PGDATABASE/PGUSER/PGPASSWORD: PostgreSQL 连接。
- Neo4j 相关变量: 连接 URI、用户、密码等。

- Agent/模型相关：以 PiCodingAgent 官方配置目录（含 AGENT_MODEL 等自动推断项）。

前端：

- VITE_API_BASE_URL：浏览器侧访问后端的基地址。

第九篇：测试、评测与达标论证

9.1 赛题指标复述

人岗匹配：关键技能匹配准确率不低于 80%。

岗位画像与学生画像：抽样评估关键信息准确率超过 90%。

判断方法：匹配从基础要求、职业技能、职业素养、发展潜力四维对比打分，按岗位维度权重综合；每维先分析再汇总。

9.2 本项目评测方法

采用手工抽样+JSONL 标注+可复算脚本：

匹配样本：20 条（data/evaluation/match.manual-samples.jsonl）。

画像样本：10 条（data/evaluation/profile.manual-samples.jsonl）。

统计：

- $\text{match_accuracy} = \text{is_correct} = \text{true} \text{ 数量} / 20$
- $\text{profile_accuracy} = \Sigma \text{correct_fields} / \Sigma \text{checked_fields}$

阈值：match_accuracy ≥ 0.80 且 profile_accuracy > 0.90。

9.3 评测结果

1. 样本规模

匹配样本：20 条

画像样本：10 条

总样本：30 条

2.统计口径

$\text{match_accuracy} = \text{is_correct} = \text{true}$ 数量 / match 总样本

$\text{profile_accuracy} = \Sigma \text{correct_fields} / \Sigma \text{checked_fields}$

阈值: $\text{match_accuracy} \geq 0.80$ 且 $\text{profile_accuracy} \geq 0.90$ 。

3.指标结果

match_accuracy : 0.8 (通过)

profile_accuracy : 0.92 (通过)

综合判定: 通过

4.未通过样本清单

4.1 匹配样本

- ✓ M003: $\text{is_correct} = \text{false}$, 备注: 缺少特征工程证据
- ✓ M007: $\text{is_correct} = \text{false}$, 备注: 沟通协同证据不足
- ✓ M010: $\text{is_correct} = \text{false}$, 备注: 自动化部署覆盖不足
- ✓ M017: $\text{is_correct} = \text{false}$, 备注: 业务抽象维度证据不足

4.2 画像样本 ($\text{correct_fields} < \text{checked_fields}$)

- ✧ P002: $\text{correct_fields} = 9 / \text{checked_fields} = 10$ (计算 $\text{accuracy} = 0.9$, 标注 $\text{accuracy} = 0.9$)
- ✧ P003: $\text{correct_fields} = 9 / \text{checked_fields} = 10$ (计算 $\text{accuracy} = 0.9$, 标注 $\text{accuracy} = 0.9$)
- ✧ P005: $\text{correct_fields} = 8 / \text{checked_fields} = 10$ (计算 $\text{accuracy} = 0.8$, 标注 $\text{accuracy} = 0.8$)
- ✧ P007: $\text{correct_fields} = 9 / \text{checked_fields} = 10$ (计算 $\text{accuracy} = 0.9$, 标注 $\text{accuracy} = 0.9$)
- ✧ P009: $\text{correct_fields} = 9 / \text{checked_fields} = 10$ (计算 $\text{accuracy} = 0.9$, 标注 $\text{accuracy} = 0.9$)
- ✧ P010: $\text{correct_fields} = 8 / \text{checked_fields} = 10$ (计算 $\text{accuracy} = 0.8$, 标注 $\text{accuracy} = 0.8$)

5.标注一致性检查

画像样本中 accuracy 与 $\text{correct_fields} / \text{checked_fields}$ 一致。

6.数据来源

match 文件: data/evaluation/match.manual-samples.jsonl

profile 文件: data/evaluation/profile.manual-samples.jsonl

9.4 复现命令

在仓库根目录执行（调用 workspace career-backend 脚本）：

```
npm run evaluation:manual
```

实际实现入口: apps/backend 内 tsxsrc/scripts/evaluation-manual.ts。

9.5 检索基线评测

```
npm run knowledge:eval
```

9.6 手工抽样评测规范

样本设计:

匹配 20 条: 覆盖不同岗位族、不同完整度简历、边界样本（极简简历、跨专业投递）。

画像 10 条: 覆盖表单与简历双入口、字段缺失、多段实习。

标注规则:

匹配: is_correct 由至少一名标注员对照四维证据判定; 争议样本由第二人仲裁。

画像: 按字段清单逐字段 correct_fields/checked_fields; 关键字段（技能列表、教育经历、项目标题等）权重可在队内规范。

一致性检查:

画像样本需校验 accuracy 与 correct_fields/checked_fields 一致。

改进闭环:

对未通过样本记录备注，归类为：数据缺失、解析错误、权重不合理、解释不充分等，进入下一迭代优先级队列。